HIGH-LEVEL DESIGN DOCUMENT

Customer Support RAG Assistant

1. System Overview

1.1 Problem Definition

Customer support operations in modern e-commerce and SaaS environments are under significant and growing strain. Support teams routinely receive thousands of queries daily, with a substantial majority of these being repetitive in nature — covering topics such as order status, return and refund policies, account issues, and general product information.

Conventional rule-based chatbots and decision-tree systems, while effective for a narrow set of predetermined scenarios, fundamentally fail to address the breadth and nuance of real human language. They are brittle, require constant manual maintenance, and cannot intelligently generalize from existing knowledge to novel query patterns.

> *Key Pain Points: High agent workload from repetitive queries, inability of traditional bots to handle natural language variation, lack of contextual awareness in automated responses, and no systematic mechanism to route complex issues to human agents.*

1.2 Scope of the System

This document describes the high-level design of an intelligent Customer Support Assistant powered by Retrieval-Augmented Generation (RAG). The system ingests structured and unstructured knowledge bases (PDF documents and CSV datasets), converts them into a searchable vector index, and uses a language model to synthesize contextually accurate, natural-language responses.

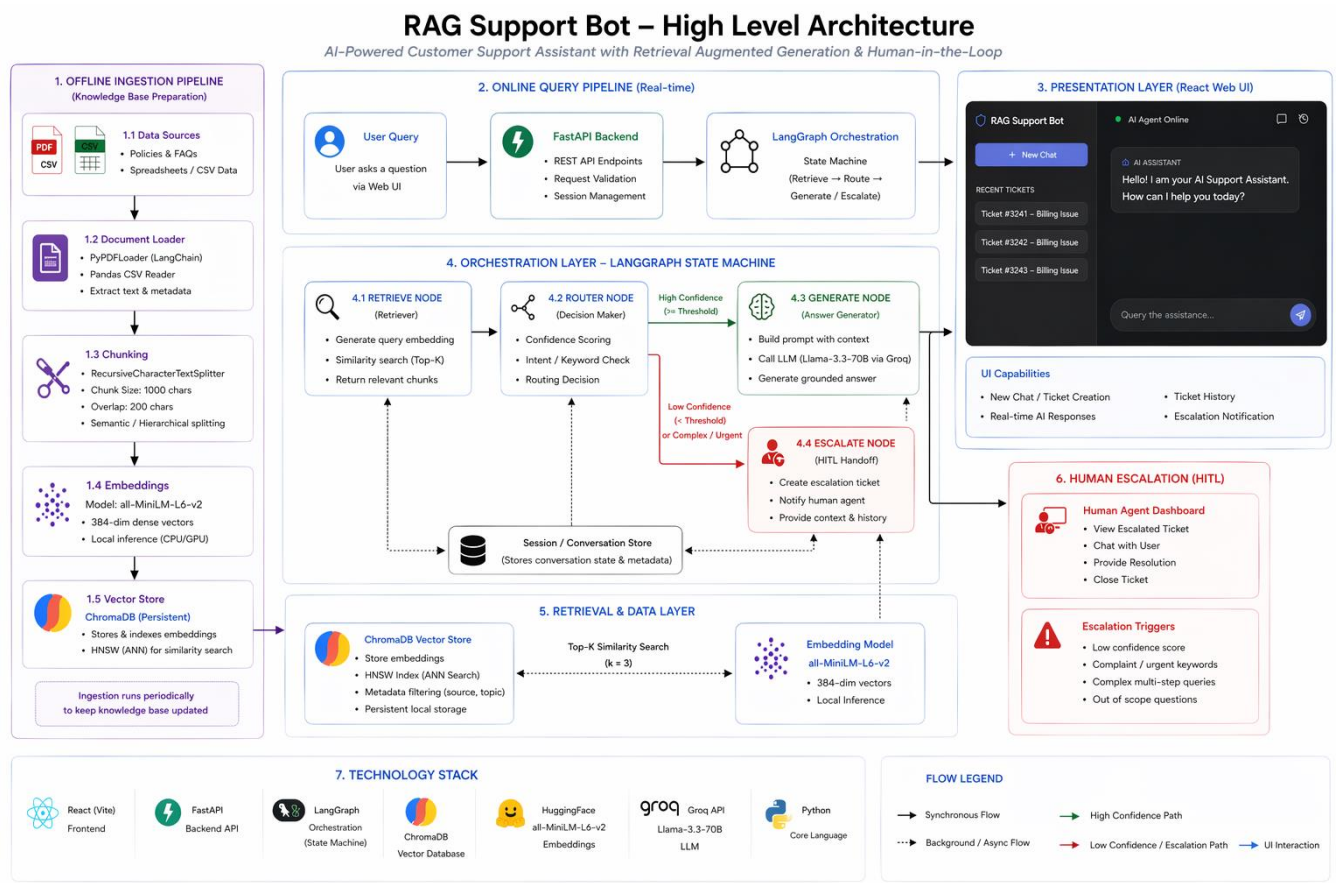
The system is scoped to cover the following functional areas:

- Document ingestion and preprocessing from PDF and CSV sources
- Semantic chunking and vector embedding of knowledge content
- Real-time semantic retrieval using similarity search
- LLM-driven response generation with contextual grounding
- Intelligent routing via a graph-based workflow engine
- Human-in-the-Loop (HITL) escalation for low-confidence or complex cases

- Web-based user interface for query submission and response display

Out of scope for this version: multi-language support, voice interface, CRM system integration, and autonomous agent action-taking (e.g., processing refunds directly).

2. Architecture Diagram



The following diagram illustrates the high-level system architecture. The system is structured across three conceptual layers: the Presentation Layer (user-facing interface), the Orchestration Layer (LangGraph-driven workflow and routing), and the Data Layer (embedding pipeline and vector store).

Architecture Layers & Components

Architectural Layer	Component	Responsibility
Presentation Layer	React Web App (UI)	Accepts user queries, displays responses, handles HITL feedback
Ingestion Layer	Document Loader + Chunker	Parses PDF/CSV, splits into semantic chunks
Embedding Layer	HuggingFace Sentence Transformers	Converts text chunks to 384-dim dense vectors
Storage Layer	ChromaDB Vector Store	Indexes and persists embeddings for similarity retrieval
Orchestration Layer	LangGraph State Machine	Manages Retrieve → Route → Generate/Escalate flow
LLM Layer	Groq API / Llama-3.3-70b	Handles generation, routing decisions, answer synthesis
HITL Layer	Human-in-the-Loop Module	Proactive escalation to human agents for edge cases
API Layer	FastAPI Backend	Exposes REST endpoints; manages async sessions

3. Component Descriptions

3.1 Document Loader

The Document Loader is the entry point for all knowledge ingestion. It is implemented using two complementary libraries to handle heterogeneous source formats:

- **PyPDFLoader (LangChain):** Handles extraction of raw text from PDF documents, including multi-page documents with mixed formatting. Preserves paragraph structure while stripping non-textual elements.
- **Pandas CSV Reader:** Processes structured training data provided in CSV format. Columns such as 'question' and 'answer' are concatenated into natural language strings before further processing.

3.2 Chunking Strategy

Raw documents are subdivided into semantically coherent chunks prior to embedding. The RecursiveCharacterTextSplitter from LangChain is employed with the following configuration:

Parameter	Value & Rationale

Chunk Size	1,000 characters — large enough to preserve multi-sentence context
Chunk Overlap	200 characters — prevents context loss at boundary transitions
Splitting Hierarchy	Splits on paragraph breaks, sentence boundaries, then words
Post-processing	Whitespace normalization and deduplication of near-identical chunks

This strategy ensures that retrieved chunks contain sufficient context for the LLM to generate coherent answers, while remaining small enough for precise similarity matching.

3.3 Embedding Model

Text chunks are converted into dense numerical representations (embeddings) using the sentence-transformers/all-MiniLM-L6-v2 model, served via the HuggingFace Transformers library.

Property	Detail
Model	sentence-transformers/all-MiniLM-L6-v2
Vector Dimensionality	384 dimensions
Inference Mode	Local CPU/GPU — no external API call required
Embedding Type	Dense semantic embeddings (cosine similarity compatible)
Throughput	~2,000 sentences/second on CPU; ~10,000/second on GPU

This model was selected for its exceptional balance between embedding quality and inference speed. It is well-suited for customer support domains where semantic proximity (not just keyword overlap) is critical for retrieval accuracy.

3.4 Vector Store

ChromaDB serves as the local vector database for storing, indexing, and querying embeddings. It provides:

- Persistent local storage — embeddings survive system restarts without re-ingestion
- Approximate Nearest Neighbor (ANN) search using HNSW indexing
- Native Python client with synchronous and asynchronous access modes
- Metadata filtering — allows filtering retrieval results by source document or topic tags

The vector store is initialized once during the ingestion pipeline and subsequently queried at inference time. For production deployments, ChromaDB can be configured to run as a persistent server, decoupling storage from the application process.

3.5 Retriever

The Retrieval Layer abstracts the vector similarity search and feeds context into the workflow engine. It is configured as follows:

- **Search Type:** Similarity search using cosine distance between query embedding and indexed chunk embeddings.
- **Top-K Results:** Default k=3 most relevant chunks are retrieved per query. This value is configurable via environment variable.
- **Context Assembly:** Retrieved chunks are ranked by similarity score and concatenated in descending relevance order before being passed to the LLM.
- **Confidence Threshold:** If the top similarity score falls below a configurable threshold (default: 0.65), the query is flagged for potential escalation.

3.6 LLM Processing Layer

The LLM layer handles all natural language generation and routing inference. It interfaces with the Groq API to access the Llama-3.3-70b model:

Attribute	Value
Model	meta-llama/Llama-3.3-70b-versatile (via Groq)
API Provider	Groq Cloud — ultra-low-latency inference infrastructure
Avg. Response Latency	< 800ms for typical support queries
Context Window	128,000 tokens — supports large document contexts
Temperature	0.2 for factual responses; 0.0 for routing decisions
System Prompt	Structured prompt with persona, constraints, and formatting rules

The LLM is invoked for two distinct purposes: (1) synthesizing a final customer-facing response from retrieved context, and (2) making routing decisions about whether a query can be answered from context or requires escalation.

3.7 Graph Workflow Engine

The orchestration backbone of the system is built on LangGraph, a stateful graph execution framework built on top of LangChain. The workflow is modeled as a directed state machine with the following nodes:

Node	Description
retrieve_node	Fetches top-k relevant chunks from ChromaDB based on the user query embedding
router_node	Evaluates retrieval confidence and query complexity to determine the processing path
generate_node	Invokes the LLM with the retrieved context to produce a structured response
escalate_node	Prepares escalation payload and triggers the HITL notification workflow
END	Terminal state — response delivered to the UI or escalation logged

Graph edges are conditional: the router_node evaluates whether retrieved context is sufficient. If yes, control passes to generate_node. If no, control passes to escalate_node. This enables true conditional branching — a capability not available in standard LangChain LCEL chains.

3.8 Routing Layer

The Routing Layer is embedded within the router_node of the LangGraph workflow. It implements a multi-signal routing decision:

- **Signal 1 — Retrieval Confidence:** Cosine similarity score from the top-retrieved chunk. Scores below 0.65 trigger escalation consideration.
- **Signal 2 — LLM Self-Assessment:** The LLM is asked to assess whether it can provide a reliable answer given the retrieved context. A structured binary output (CAN_ANSWER / ESCALATE) is returned.
- **Signal 3 — Keyword Heuristics:** Explicit trigger phrases (e.g., 'speak to a human', 'urgent', 'complaint') bypass the confidence check and route directly to escalation.

The combination of these three signals ensures both false-positive and false-negative escalation rates are minimized, balancing automation throughput with escalation quality.

3.9 HITL Module

The Human-in-the-Loop (HITL) module provides a structured mechanism for escalating queries that exceed the system's confidence threshold or involve sensitive situations. The HITL design follows a proactive consent model:

- When the router determines escalation is warranted, the system first informs the user that the query may require human assistance.
- The user is presented with a binary choice: proceed with the AI-generated best-effort response, or escalate to a human agent.
- If escalation is chosen, the conversation context, retrieved chunks, and confidence scores are packaged into an escalation payload.
- In the current implementation, the escalation simulates routing to a human agent via a structured log entry. In production, this connects to a ticketing system (e.g., Zendesk, Freshdesk).

4. Data Flow

4.1 Document Ingestion Lifecycle (Offline)

The ingestion pipeline is a one-time (or scheduled) offline process that transforms raw knowledge documents into a queryable vector index. The steps are:

Step	Stage	Description
Step 1	Document Upload	User uploads PDF/CSV via the React UI or CLI ingestion script
Step 2	Format Detection	System detects file type and routes to PyPDFLoader or Pandas reader
Step 3	Text Extraction	Raw text is extracted, preserving structural metadata (page numbers, section headers)
Step 4	Chunking	RecursiveCharacterTextSplitter subdivides text into 1000-char chunks with 200-char overlap
Step 5	Embedding	Each chunk is passed to the HuggingFace embedding model, producing a 384-dim vector
Step 6	Indexing	Chunk text, embedding vector, and metadata are persisted to ChromaDB
Step 7	Confirmation	Ingestion summary (chunk count, document ID) is returned to the user

4.2 Query Processing Lifecycle (Online / Real-Time)

Every user query undergoes the following end-to-end processing pipeline:

Step	Stage	Description
Step 1	Query Submission	User submits natural language query via the React chat interface
Step 2	Query Embedding	FastAPI backend embeds the query using the same HuggingFace model used at ingestion
Step 3	Vector Retrieval	ChromaDB performs cosine similarity search, returning top-3 chunks with similarity scores
Step 4	Context Assembly	Chunks are ranked and assembled into a structured context string
Step 5	Routing Decision	LangGraph router_node evaluates confidence signals and determines processing path
Step 6a	Generation (if confident)	LLM synthesizes a response grounded in retrieved context; response is structured and cited
Step 6b	Escalation (if not confident)	HITL module presents escalation prompt to user; logs context for human agent
Step 7	Response Delivery	Final answer (or escalation acknowledgment) is streamed to the React UI via WebSocket

5. Technology Choices & Justification

5.1 ChromaDB — Vector Store

- **Why ChromaDB:** ChromaDB was selected over alternatives (Pinecone, Weaviate, FAISS) for its zero-infrastructure-overhead local deployment model. It requires no cloud credentials, no containerized server process, and no network configuration — making it ideal for development and small-to-medium production deployments.

Comparison	Rationale
vs. Pinecone	Pinecone requires cloud setup, API keys, and incurs ongoing costs. ChromaDB is free and local.
vs. FAISS	FAISS is index-only; ChromaDB includes a full document store with metadata filtering.
vs. Weaviate	Weaviate requires Docker and GraphQL schema setup. ChromaDB is simpler and faster to iterate on.
vs. Qdrant	Qdrant requires a running server process. ChromaDB can run fully in-process.

5.2 LangGraph — Workflow Orchestration

LangGraph was selected as the workflow engine because customer support flows are inherently non-linear. Standard LangChain LCEL (LangChain Expression Language) pipelines are linear and cannot express:

- Conditional branching based on retrieval confidence
- Feedback loops (e.g., retry with a different retrieval strategy if confidence is low)
- Human-in-the-loop pauses that wait for user confirmation before proceeding
- State persistence across multi-turn conversations

LangGraph addresses all of these with its directed graph abstraction, where nodes are processing steps and edges are conditional transitions. This maps directly to the support workflow's Retrieve → Route → Generate/Escalate structure.

5.3 Groq / Llama-3.3-70b — LLM

The LLM selection was driven by three criteria: inference speed, reasoning quality, and cost efficiency.

Criterion	Justification
Speed	Groq's LPU (Language Processing Unit) infrastructure delivers sub-second inference, critical for real-time support UX.
Reasoning Quality	Llama-3.3-70b achieves near-GPT-4 performance on instruction-following and multi-step reasoning benchmarks.
Cost	Groq API pricing is significantly lower per token than OpenAI GPT-4o for equivalent performance.
Open Weights	Llama-3 is an open-weights model, enabling future self-hosting to eliminate API dependency.

5.4 Additional Technology Stack

Tool / Library	Role & Justification
FastAPI	Asynchronous Python web framework; handles concurrent chat sessions with minimal overhead via asyncio.
React (TypeScript)	Component-based UI framework; enables real-time WebSocket message rendering and responsive chat layout.
HuggingFace Transformers	Provides the local embedding model inference pipeline; avoids external API dependency for embeddings.
LangChain	Core abstraction layer for document loaders, text splitters, and retriever interfaces; reduces boilerplate.

Uvicorn	ASGI server for FastAPI; supports HTTP/2 and WebSocket protocols for low-latency streaming.
Python-dotenv	Manages environment variables (API keys, config) cleanly across deployment environments.

6. Scalability Considerations

6.1 Handling Large Document Volumes

The system is designed to handle knowledge bases of varying sizes without degradation. Key strategies include:

- **Batch Ingestion:** Large document sets (e.g., 26,000+ CSV rows) are processed in configurable batches of 500 records. This prevents memory overflow during the embedding step and allows for incremental ingestion with progress tracking.
- **Incremental Updates:** ChromaDB supports upsert operations, allowing new documents to be added to the existing index without full re-ingestion. Document IDs are hashed from content to detect duplicates.
- **Async Ingestion:** The ingestion pipeline is decoupled from the query-serving API. A background task queue (e.g., Celery or FastAPI BackgroundTasks) processes uploads asynchronously so the API remains responsive during ingestion.

6.2 Handling Increasing Query Load

- **Horizontal Scaling:** The FastAPI backend is stateless (session state is managed client-side or in Redis). Multiple instances can be deployed behind a load balancer (e.g., Nginx, AWS ALB) to distribute query load.
- **ChromaDB Server Mode:** For high-concurrency deployments, ChromaDB can be switched from in-process mode to HTTP server mode, allowing multiple application instances to share a single vector index.
- **Caching Layer:** Frequently asked queries (identified via query hash matching) can be served from a Redis cache, bypassing the retrieval and LLM pipeline entirely for repeated questions.

6.3 Latency Management

Component	Target Latency / Strategy
Embedding Latency	< 50ms per query (CPU) — HuggingFace model runs locally, no network round-trip

Retrieval Latency	< 30ms — ChromaDB HNSW index provides near-constant-time lookup
LLM Inference Latency	< 800ms — Groq LPU delivers ultra-fast token generation
End-to-End P95 Latency	< 1.2 seconds for standard queries; < 2s for complex multi-chunk synthesis
Streaming	WebSocket streaming of LLM tokens provides perceived responsiveness before completion

6.4 Fault Tolerance & Reliability

- **Retry Logic:** All external API calls (Groq) are wrapped in exponential backoff retry logic with a maximum of 3 attempts before failing gracefully.
- **Fallback Responses:** If the LLM API is unavailable, the system falls back to returning the top retrieved chunk directly, with a disclaimer that a full AI response is temporarily unavailable.
- **Health Endpoints:** FastAPI exposes /health and /ready endpoints for load balancer health checks and Kubernetes readiness probes.
- **Logging & Observability:** Structured JSON logging is implemented throughout the pipeline. Retrieval scores, routing decisions, and LLM response metadata are logged for monitoring and debugging.

7. Assumptions & Constraints

The following assumptions and constraints govern the design decisions documented in this HLD:

Assumption / Constraint	Detail
Knowledge Base Format	Source documents are provided in PDF or structured CSV format. HTML, Word, and other formats require a pre-conversion step.
Language	The system is designed for English-language queries and documents. Multilingual support is out of scope for v1.
Connectivity	The Groq API requires internet connectivity. An offline fallback (local Llama via Ollama) is planned for v2.
Data Privacy	No user query data is persisted beyond the active session. Conversation logs are opt-in and anonymized.
Human Agent Simulation	HITL escalation simulates agent handoff via structured logs. Live integration with ticketing systems is a v2 feature.
Knowledge Freshness	The vector index reflects the knowledge base at ingestion time. Automated re-ingestion on document updates is a v2 feature.

8. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — a pattern that grounds LLM responses in retrieved documents rather than relying on parametric memory.
LLM	Large Language Model — a neural network trained on large text corpora capable of natural language generation and reasoning.
HITL	Human-in-the-Loop — a design pattern where human judgment is incorporated into an automated workflow at defined decision points.
Vector Embedding	A dense numerical representation of text that encodes semantic meaning, enabling similarity-based search.
ChromaDB	An open-source vector database optimized for embedding storage and similarity retrieval.
LangGraph	A library for building stateful, graph-based LLM workflows with support for conditional branching and cycles.
Groq	A cloud AI inference provider using proprietary LPU hardware to achieve ultra-low-latency model serving.
HNSW	Hierarchical Navigable Small World — an approximate nearest neighbor algorithm used by ChromaDB for fast vector search.
ANN	Approximate Nearest Neighbor — a class of algorithms that find near-optimal nearest neighbors in sub-linear time.
FastAPI	A modern, high-performance Python web framework for building APIs with async support and automatic OpenAPI docs.